DOCUMENTATION TECHNIQUE PRINCIPALE

1. PRESENTATION GENERALE DU PROJET

Le projet DigiMarket s'inscrit dans une stratégie de diversification de l'entreprise vers la vente en ligne de matériel informatique. L'objectif est de fournir une solution robuste et scalable via une API REST permettant la gestion du catalogue produits, des utilisateurs et du cycle de vie des commandes.

STACK TECHNIQUE

Composant	Technologie	Détails
Langage	Python 3.13	Utilisation des dernières fonctionnalités de typage et performance.
Framework	Flask	Architecture modulaire basée sur les Blueprints.
ORM	SQLAlchemy	Mapping objet-relationnel pour la couche de données.
Base de Données	SQLite	Fichier local digimarket.db pour la persistance.
Authentification	JWT & Werkzeug	Sécurisation par jetons et hachage de mots de passe.

2. ARCHITECTURE DU SYSTEME (MODELE MVC)

L'application suit le pattern **MVC (Modèle-Vue-Contrôleur)** afin de découpler la logique métier de la présentation et de l'accès aux données.

- **Modèles (M) :** Définis dans app/database/model.py. SQLAlchemy est utilisé pour traduire les classes Python (Utilisateur, Produit, Commande, LigneCommande) en tables relationnelles, gérant nativement les contraintes d'intégrité et les relations (clés étrangères).
- **Contrôleurs (C) :** La logique est segmentée en **Blueprints** Flask pour une maintenance facilitée :
- auth_bp : Gère le cycle de vie des sessions et la sécurité.
- products_bp : Administre le catalogue et les stocks.
- orders_bp : Orchestre la création et le suivi des commandes.
- **Vues (V) :** L'architecture gère deux types de sorties :
- **REST API :** Réponses formatées en JSON pour les clients programmatiques.
- **Interface Web :** Le blueprint web_bp (dans app/web/routes.py) utilise le moteur de template **Jinja2** pour servir des pages HTML dynamiques (catalog.html, cart.html, etc.).

3. SECURITE ET PROTECTION DES DONNEES

AUTHENTIFICATION JWT

Le système utilise des jetons **JSON Web Tokens (JWT)** via la bibliothèque pyjwt.

- **Génération :** La fonction generate_json_token utilise l'algorithme HS256 et une SECRET_PHRASE statique (d3fb12750c2eff92120742e1b334479e) pour signer les payloads contenant le login et l'expiration (expire_time).
- **Vérification :** La fonction verify_token décode et valide l'intégrité du jeton avant chaque requête protégée.

HACHAGE DES MOTS DE PASSE

Pour prévenir toute compromission, les mots de passe sont hachés via werkzeug.security. Les fonctions generate_password_hash et check_password_hash sont systématiquement appelées lors de l'inscription et de l'authentification.

CONTROLE D'ACCES (RBAC)

Le décorateur @requires_authorization assure la sécurité au niveau des routes. Il extrait l'attribut role de l'utilisateur en base via la fonction get_user_attribute_in_db (basée sur le login présent dans le token). L'accès aux méthodes sensibles (POST, PUT, DELETE sur les produits) est strictement réservé au rôle admin.

4. MODELISATION DE LA BASE DE DONNEES

Table	Colonne	Type	Nullable	Contraintes
Utilisateur	id	Integer	No	PK
	email	String(120)	No	Unique
	mot_de_passe	String(200)	No	
	nom	String(100)	No	
	role	String(50)	No	Default: 'client'
	date_creation	DateTime	Yes	
Produit	id	Integer	No	PK

	nom	String(200)	No	
	description	Text	Yes	
	categorie	String(100)	Yes	
	prix	Float	No	
	quantite_stock	Integer	No	Default: 0
	date_creation	DateTime	Yes	
Commande	id	Integer	No	PK
	utilisateur_id	Integer	No	FK (utilisateur.id)
	date_commande	DateTime	Yes	
	adresse_livraison	String(500)	Yes	
	statut	String(50)	No	Default: 'en_attente'
LigneCommande	id	Integer	No	PK
	commande_id	Integer	No	FK (commande.id)
	produit_id	Integer	No	FK (produit.id)
	quantite	Integer	No	
	prix_unitaire	Float	No	

5. SPECIFICATIONS DE L'API REST

5.1. AUTHENTIFICATION

INSCRIPTION (POST /AUTH/REGISTER)

- **Rôle :** Public
- Requête :

```
{
  "EMAIL": "TESTED_CLIENT_1@TEST.NET",
  "MOT_DE_PASSE": "BLENT",
  "NOM": "TESTED CLIENT 1",
  "ROLE": "CLIENT"
}
```

- Réponse :


```
[
  {
    "date_creation": "wed, 22 apr 2026 19:51:12 gmt",
    "email": "tested_client_1@test.net",
    "id": 14,
    "mot_de_passe": "blent",
    "nom": "tested client 1",
    "role": "client"
  }
]
```


CONNEXION (POST /AUTH/LOGIN)

- **Rôle :** Public
- Requête :


```
{
  "email": "customer@test.net",
  "password": "blent"
}
```


- Réponse :


```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```


5.2. CATALOGUE PRODUITS

LISTE COMPLETE (GET /API/PRODUITS)

- **Paramètres :** keywords (optionnel)
- Réponse :


```
[
```

```
 {
   "categorie": "Stockage",
   "date_creation": "2026-04-07 17:31:38.000000",
   "description": "High-performance Stockage hardware...",
   "id": 2,
   "nom": "Stockage Corsair - SSD NVMe 1To",
   "prix": 147.6,
   "quantite_stock": 3
 }
]
```


DETAILS D'UN PRODUIT (GET /API/PRODUITS/{ID})

- Réponse :


```
[
 {
   "categorie": "Stockage",
   "id": 2,
   "nom": "Stockage Corsair - SSD NVMe 1To",
   "prix": 147.6,
   "quantite_stock": 3,
   "date_creation": "Tue, 07 Apr 2026 17:31:38 GMT"
 }
]
```


AJOUT D'UN PRODUIT (POST /API/PRODUITS)

- **Rôle :** Admin uniquement
- Requête :


```
{
 "nom": "Mouse wireless Microsoft",
 "description": "Black, rechargeable, USB-C radio emitter",
 "categorie": "Accessories",
 "prix": 45.0,
 "quantite_stock": 30
}
```

- Réponse :

```
[
  {
    "categorie": "Accessories",
    "id": 56,
    "nom": "Mouse wireless Microsoft",
    "prix": 45.0,
    "quantite_stock": 30
  }
]
```

MISE A JOUR D'UN PRODUIT (PUT /API/PRODUITS/{ID})

- **Rôle :** Admin uniquement
- Requête :

```
{
  "nom": "Mouse Microsoft XYZ123",
  "prix": 40.0,
  "categorie": "Souris"
}
```

- Réponse :

```
[
  {
    "categorie": "Souris",
    "id": 56,
    "nom": "Mouse Microsoft XYZ123",
    "prix": 40.0,
    "quantite_stock": 30
  }
]
```

SUPPRESSION D'UN PRODUIT (DELETE /API/PRODUITS/{ID})

- **Rôle :** Admin uniquement
- Réponse :

```
{
  "info": "product with id 56 has been deleted."
}
```

5.3. GESTION DES COMMANDES

LISTE DES COMMANDES (GET /API/COMMANDES)

- **Rôle :** Client (propres commandes) / Admin (toutes)
- Réponse :

```
[
  {
    "adresse_livraison": "123, rue de la paix, paris",
    "date_commande": "wed, 22 apr 2026 19:51:12 gmt",
    "id": 11,
    "statut": "en_attente",
    "utilisateur_id": 11
  }
]
```

CREATION D'UNE COMMANDE (POST /API/COMMANDES)

- Requête :

```
{
  "adresse_livraison": "123, rue de la paix, paris"
}
```

- Réponse :

```
[
  {
```

```
    "adresse_livraison": "123, rue de la paix, paris",
    "id": 11,
    "statut": "en_attente",
    "utilisateur_id": 11
  }
]
```


MISE A JOUR DU STATUT (PATCH /API/COMMANDES/{ID})

- **Rôle :** Admin uniquement
- Requête :


```
{
  "statut": "validée"
}
```


- Réponse :


```
[
  {
    "id": 12,
    "statut": "validée",
    "adresse_livraison": "123, rue de la paix, paris"
  }
]
```


DETAILS DES LIGNES DE COMMANDE (GET /API/COMMANDES/{ID}/LIGNES)

- Réponse :


```
[
  {
    "commande_id": 11,
    "id": 21,
    "prix_unitaire": 40.0,
    "produit_id": 56,
    "quantite": 2
  }
```

```
]
```

6. MATRICE DE TRAÇABILITE

ID Exigence	Description	Implémentation (Endpoint)
REQ-AUTH-01	Inscription et hachage PW	POST /auth/register
REQ-AUTH-02	Connexion et émission JWT	POST /auth/login
REQ-PROD-01	Consultation catalogue / Recherche	GET /api/produits
REQ-PROD-02	Fiche produit détaillée	GET /api/produits/{id}
REQ-ADMIN-01	Création de produit (RBAC Admin)	POST /api/produits
REQ-ADMIN-02	Mise à jour de produit (RBAC Admin)	PUT /api/produits/{id}
REQ-ADMIN-03	Suppression de produit (RBAC Admin)	DELETE /api/produits/{id}
REQ-ORDER-01	Création de commande client	POST /api/commandes
REQ-ORDER-02	Consultation historique commandes	GET /api/commandes
REQ-ORDER-03	Détail des articles commandés	GET /api/commandes/{id}/lignes
REQ-ORDER-04	Validation stock et statut	PATCH /api/commandes/{id}

7. GUIDE D'INSTALLATION ET D'UTILISATION

PREREQUIS

- Python 3.13
- Git

PROCEDURE

1. **Clonage :** git clone https://github.com/JeanRosselVallee/BlentPythonProject.git
2. **Environnement :** python -m venv .venv puis activation (ex: ..venv\Scripts\activate sur Windows).
3. **Dépendances :** pip install -r requirements.txt
4. **Initialisation :** python run.py (crée digimarket.db).
5. **Peuplement :** python -m app.database.populate (injecte 10 enregistrements par table).

TESTS

Exécuter la suite de tests complète via : python -m pytest

8. ANNEXES TECHNIQUES

GESTION DU STOCK

La logique de validation de stock intervient lors de l'appel à POST /api/commandes/{id}/lignes. Le système compare la quantité demandée au champ quantite_stock du produit. En cas d'insuffisance, l'API renvoie une erreur 400 avec un message explicite.

- **Exemple d'erreur :** {"error": "200 > stock of 30 for product id 56."}

LOGS

Le système de logging est configuré via pytest.ini pour produire des logs structurés dans tests/pytest.log.

- **Format :** %Y-%m-%d %H:%M:%S %(levelname)s %(message)s
- **Exemple :** 2026-04-22 21:51:15 INFO User customer@test.net was created in DB.